OM DIGITAL E-SEVA KENDRA

Authentication & Login System Technical Audit Report | Date: 2026-10-29

CONFIDENTIAL & SENSITIVE — INTERNAL AUDIT ONLY

This document details the complete technical architecture, endpoints, database schemas, and confirmed bugs in the OM Digital E-Seva Kendra login system. Passwords, hashes, and API keys are redacted for security.

1. Executive Summary & Architecture Overview

The OM Digital E-Seva Kendra software uses a decoupled client-server architecture. Authentication is managed via a Node.js/Express backend connected to a TiDB Cloud MySQL database, issuing 7-day HMAC-SHA256 JWT tokens. The client is a React Single Page Application (SPA) with role-based routing (Admin, Operator, Sub-Agent) and granular feature permissions.

- **Database Engine:** TiDB Cloud (MySQL v8.0 compatible, AWS Asia-Southeast1 cluster)
- **Frontend Framework:** React 19 + TypeScript + Tailwind CSS
- **Backend Runtime:** Express 4.21 on Node.js with tsx compiler
- **Session Strategy:** Dual Token Architecture: JWT Bearer Tokens (7-day validity) + Fallback User Identity Headers
- **Password Hashing:** Bcrypt with 10 salt rounds (with automatic legacy hash upgrade on login)

2. Frontend Login Flow & Implementation

2.1 Primary Login Component (/src/components/LoginPage.tsx)

The login form captures either a Username or Mobile Number along with the Password. On submission, it executes a POST request to /api/auth/login. On success, it persists the JWT token and notifies the root App component.

```
// Login Submission in LoginPage.tsx
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  const inputVal = usernameOrMobile.trim();
  const res = await fetch('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ usernameOrMobile: inputVal, password: password }),
  });
  const data = await res.json();
  if (!res.ok) throw new Error(data.error || 'Invalid credentials');
  if (data.token) localStorage.setItem('om_eseva_token', data.token);
  localStorage.setItem('mbocw_last_login_time', new Date().toISOString());
  onLoginSuccess(data.user, true);
};
```

2.2 State Management & Route Protection (/src/App.tsx)

The root component initializes currentUser from storage (sessionStorage or localStorage). When currentUser is null, it unconditionally returns <LoginPage />. Once logged in, it mounts <Navbar />, <Sidebar />, and the active dashboard tab matching the user role.

3. Backend Authentication & API Layer

3.1 Global Authentication Middleware (/server.ts - jwtAuthMiddleware)

All routes under /api are intercepted by jwtAuthMiddleware. It enforces token validation, whitelists public routes (/api/auth/login, /api/health, etc.), and validates user activity status in MySQL.

```
// JWT & Session Guard in server.ts
async function jwtAuthMiddleware(req, res, next) {
  const fullPath = (req.originalUrl || req.path || '').split('?')[0];
  if (isPublicRoute(fullPath)) return next();

  let token = '';
  const authHeader = req.headers['authorization'];
  if (authHeader && authHeader.startsWith('Bearer ')) {
    token = authHeader.substring(7).trim();
  }

  if (token) {
    try {
      const decoded = jwt.verify(token, JWT_SECRET);
      req.user = decoded;
      return next();
    } catch (err) {
      return res.status(401).json({ error: 'Session expired. Please login again.', code:
'AUTH_TOKEN_INVALID' });
    }
  }

  // Fallback: Verify x-user-id / x-user-username headers against MySQL database
  const userIdHeader = req.headers['x-user-id'];
  const allUsers = await getMySQLUsers();
  const existing = allUsers.find(u => String(u.id) === String(userIdHeader));
  if (!existing || existing.status === 'disabled') {
    return res.status(401).json({ error: 'Unauthorized: Invalid user session', code:
'AUTH_TOKEN_REQUIRED' });
  }
  req.user = existing;
  return next();
}
```

3.2 Login Endpoint (/server.ts - POST /api/auth/login)

Receives { usernameOrMobile, password }. Invokes authenticateMySQLUser(). Verifies user status is not disabled, logs the LOGIN event to user_activity_logs, signs a 7-day JWT token, and returns user payload.

3.3 Role Authorization Guard (requireAdmin)

Guards administrative endpoints (/api/users, /api/settings, /api/activity-logs). Checks if req.user.role.toUpperCase() === "ADMIN" or username matches system admins (om, admin).

4. Database Schema & Query Logic

4.1 Users Table Definition (TiDB Cloud MySQL)

The users table stores core credentials and granular permission toggles for operators and agents.

```
CREATE TABLE IF NOT EXISTS `users` (
  `id` VARCHAR(64) NOT NULL PRIMARY KEY,
  `username` VARCHAR(64) NOT NULL UNIQUE,
  `password` VARCHAR(255) NOT NULL,
  `password_hash` VARCHAR(255) DEFAULT NULL,
  `mobile` VARCHAR(20) NOT NULL UNIQUE,
  `name` VARCHAR(128) NOT NULL,
  `email` VARCHAR(128) DEFAULT NULL,
  `role` VARCHAR(32) NOT NULL DEFAULT 'OPERATOR',
  `status` VARCHAR(32) NOT NULL DEFAULT 'ACTIVE',
  `photo_url` MEDIUMTEXT DEFAULT NULL,
  `can_register` TINYINT(1) NOT NULL DEFAULT 1,
  `can_renew` TINYINT(1) NOT NULL DEFAULT 1,
  `can_claim` TINYINT(1) NOT NULL DEFAULT 1,
  `can_export` TINYINT(1) NOT NULL DEFAULT 1,
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `last_login` DATETIME DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

4.2 Query & Password Verification (/src/db/mysql.ts)

authenticateMySQLUser queries by username (case-insensitive) or exact mobile. It checks bcrypt.compare(password, storedPassword). If legacy unhashed plain password matches, it transparently hashes it with bcrypt and updates the database.

```
// Query execution in mysql.ts
const [rows] = await p.query(
  `SELECT * FROM users WHERE LOWER(username) = ? OR mobile = ?`,
  [searchStr, cleanMobile]
);
for (const dbUser of rows) {
  let isMatch = false;
  if (dbUser.password.startsWith('$2a$') || dbUser.password.startsWith('$2b$')) {
    isMatch = await bcrypt.compare(pass, dbUser.password);
  } else {
    isMatch = (pass === dbUser.password);
    if (isMatch) {
      const newHash = await bcrypt.hash(pass, 10);
      await p.query('UPDATE users SET password = ? WHERE id = ?', [newHash, dbUser.id]);
    }
  }
  if (isMatch) return mapDbUser(dbUser);
}
```

5. Confirmed Bugs & Security Vulnerabilities

CRITICAL BUG 1: Session Verification Omits Authorization Header

In src/App.tsx line 280, the mount verification call to /api/auth/me only sends x-user-id and x-user-username, but omits Authorization: Bearer <token>. Because /api/auth/me is not in the jwtAuthMiddleware public whitelist, any delay or lookup failure causes HTTP 401, triggering handleLogout() and logging users out randomly on page refresh.

CRITICAL BUG 2: Role and Status Casing Discrepancies

MySQL schema sets role DEFAULT "OPERATOR" and status DEFAULT "ACTIVE" (uppercase). In React (App.tsx), code strictly checks currentUser.role === "admin" and status === "active" (lowercase). Any uppercase record silently breaks admin menus and falsely evaluates as disabled.

BUG 3: Strict Mobile Format Matching Fails on Standard Inputs

In mysql.ts line 1666, cleanMobile is compared directly against the database string without normalizing spaces, country code (+91), or leading zeros. Users typing standard formatted numbers (+91 XXXXX XXXXX) fail login despite correct passwords.

BUG 4: Token Storage Desynchronization

LoginPage.tsx unconditionally writes the JWT token to localStorage, while App.tsx writes user profile to sessionStorage if Remember Me is not selected. Closing the browser clears the user session but leaves a stale JWT token in localStorage.

6. Actionable Fixes & Recommendations

- **Fix Session Sync Header in App.tsx:** Change fetch("/api/auth/me", { headers: ... }) to use headers: getAuthHeaders(), ensuring Bearer JWT token is always supplied.
- **Normalize Roles & Status to Lowercase:** In mysql.ts (authenticateMySQLUser & getMySQLUsers), ensure role.toLowerCase() and status.toLowerCase() are always returned so frontend role checks never fail.
- **Normalize Mobile Numbers on Query:** Clean input mobile with cleanMobile.replace(/\D/g, "").slice(-10) to match 10-digit Indian numbers regardless of prefix.
- **Synchronize Token Storage:** Write token to sessionStorage when Remember Me is false, and to localStorage when true, matching user session lifecycle.
- **Implement Refresh Token Endpoint:** Add a token refresh mechanism so active users are not logged out abruptly after 7 days.